

Laporan Praktikum Kontrol Cerdas

Nama : Fryscha Febryanti Puspitasari

NIM : 224308008

Kelas : TKA 6A

Akun Github (Tautan) : <https://github.com/Fryscha>

Student Lab Assistant : Yulia Brilianty

1. Judul Percobaan

Machine Learning for Control Systems

2. Tujuan Percobaan

Percobaan ini dilakukan dengan tujuan sebagai berikut.

- Memahami dasar-dasar Machine Learning dalam sistem kendali.
- Mengelola dataset dan melakukan pelatihan model sederhana.
- Mengimplementasikan model Machine Learning sederhana untuk klasifikasi objek.
- Mengintegrasikan model Machine Learning dengan Computer Vision untuk deteksi objek.
- Menggunakan Scikit-learn untuk membuat model Machine Learning dasar.

3. Landasan Teori

Machine Learning (ML) adalah salah satu percabangan dari kecerdasan buatan yang berfokus pada pengembangan algoritma dan teknik yang memungkinkan untuk belajar dan membuat prediksi atau keputusan berdasarkan data. Prinsip dasar dari Machine Learning terdiri dari data, model, fungsi kerugian (loss function), dan optimisasi. Metode Machine Learning terbagi menjadi tiga bagian yaitu Supervised Learning, Unsupervised Learning, dan Reinforcement Learning. Peran Machine Learning dalam sistem kendali yakni menggunakan algoritma untuk menganalisis data dan membuat prediksi tanpa pemrograman eksplisit. Hal ini dapat memungkinkan sistem untuk belajar dari data dan meningkatkan kinerjanya seiring waktu.

Python adalah salah satu bahasa pemrograman yang digunakan dalam pengembangan sistem kontrol cerdas. Bahasa pemrograman ini sangat diminati oleh pengguna karena

kemudahan penggunaannya dan banyaknya pustaka yang tersedia. Selain itu, terdapat OpenCV sebagai pustaka open-source yang digunakan dalam pengolahan citra dan Computer Vision.

GitHub yaitu salah satu platform untuk version control dalam mengembangkan perangkat lunak. Dalam kontrol cerdas, platform ini mampu melakukan kolaborasi antar peneliti dan pengembang untuk berbagi dataset dan model yang digunakan dalam percobaan atau penelitian, mengelola kode sumber secara efisien, dan melacak perubahan dan revisi dalam proyek.

Kaggle merupakan salah satu platform yang menyediakan berbagai dataset yang dapat digunakan untuk pelatihan dan pengujian model sistem kontrol cerdas. Manfaat dari Kaggle yakni akses ke dataset yang beragam dan berkualitas tinggi, kompetisi yang dapat meningkatkan keterampilan dalam penerapan Machine Learning, juga komunitas yang aktif untuk berbagi pengetahuan dan teknik analisis data.

Dataset adalah komponen penting dalam analisis data. Pemilihan dataset harus mempertimbangkan keakuratan, kelengkapan, dan relevansi. Pengelolaan dataset merupakan langkah penting dalam pengembangan model Machine Learning. Dataset harus dikumpulkan, dibersihkan, dan diproses sebelum digunakan untuk pelatihan. Langkah-langkah yang umum dilakukan meliputi pengumpulan data yang relevan dari berbagai sumber, seperti sensor, database, atau file; Menghapus data yang tidak valid, mengisi nilai yang hilang, dan menghapus duplikasi; Terakhir, memisahkan dataset menjadi data pelatihan, validasi, dan pengujian untuk memastikan model dapat dievaluasi dengan baik.

4. Analisis dan Diskusi

Analisis Hasil:

a. Bagaimana performa model dalam mendeteksi warna?

Jawab:

Model yang digunakan untuk mendeteksi warna, seperti Convolutional Neural Networks (CNN), menunjukkan performa yang baik dalam mengidentifikasi dan mengklasifikasikan warna dalam citra. Penelitian yang dilakukan oleh Pengaruh Dataset terhadap Performa Convolutional Neural Network menunjukkan bahwa model CNN dapat mencapai akurasi yang tinggi dalam mendeteksi warna, dengan kemampuan untuk mengenali variasi warna yang kompleks dalam berbagai kondisi pencahayaan. Model ini memanfaatkan fitur visual dari citra, seperti tekstur dan pola warna, untuk meningkatkan akurasi klasifikasi. Hasil

penelitian menunjukkan bahwa model dapat mencapai akurasi di atas 90% dalam pengenalan warna, tergantung pada kualitas dan variasi dataset yang digunakan.

b. Bagaimana perbedaan akurasi jika jumlah dataset ditambah?

Jawab:

Penambahan jumlah dataset secara signifikan dapat meningkatkan akurasi model klasifikasi. Dalam penelitian yang sama, penulis mencatat bahwa dengan meningkatkan jumlah data pelatihan, model dapat belajar lebih banyak pola dan variasi dalam data, yang berkontribusi pada peningkatan akurasi. Misalnya, ketika dataset diperluas dari 1.000 menjadi 10.000 gambar, akurasi model meningkat dari 75% menjadi 92%. Hal ini menunjukkan bahwa model lebih mampu generalisasi dan mengurangi overfitting ketika dilatih dengan dataset yang lebih besar dan beragam. Penelitian ini menekankan pentingnya kualitas dan kuantitas data dalam pengembangan model yang efektif.

c. Bagaimana cara meningkatkan kinerja model klasifikasi?

Jawab:

Cara meningkatkan kinerja model klasifikasi antara lain:

- **Augmentasi Data**
Menggunakan teknik augmentasi data seperti rotasi, pemotongan, dan perubahan warna untuk meningkatkan variasi dalam dataset pelatihan.
- **Penggunaan Model Transfer Learning**
Memanfaatkan model yang telah dilatih sebelumnya pada dataset besar dan melakukan fine-tuning untuk tugas spesifik dapat meningkatkan akurasi.
- **Penanganan Ketidakseimbangan Data**
Menggunakan teknik seperti SMOTE (Synthetic Minority Over-sampling Technique) untuk menangani ketidakseimbangan dalam dataset, sehingga model tidak bias terhadap kelas mayoritas.
- **Optimasi Hyperparameter**
Melakukan pencarian hyperparameter untuk menemukan kombinasi terbaik dari parameter model, seperti learning rate, jumlah lapisan, dan ukuran batch.
- **Regularisasi**
Menggunakan teknik regularisasi seperti L1 atau L2 untuk mencegah overfitting dan meningkatkan generalisasi model.

Diskusi:

a. Apa keuntungan Machine Learning dibandingkan metode berbasis aturan (rule-based)?

Jawab:

Keuntungan Machine Learning dibandingkan metode berbasis aturan (rule-based) sebagai berikut.

- Machine Learning dapat belajar dari data dan beradaptasi dengan perubahan lingkungan atau pola data, sedangkan metode berbasis aturan memerlukan pemrograman ulang untuk setiap perubahan.
- Machine Learning mampu menganalisis dan memproses volume data yang sangat besar dan kompleks, sehingga sulit dilakukan jika dilakukan berbasis aturan.
- Dengan algoritma yang terus berjalan dari data baru, Machine Learning dapat meningkatkan akurasi prediksi dan keputusan seiring waktu, sedangkan metode berbasis aturan cenderung statis.
- Machine Learning dapat menemukan pola yang tidak terlihat dalam data, yang mungkin tidak dapat ditangkap oleh aturan yang ditetapkan secara manual.

b. Bagaimana ML dapat diintegrasikan lebih lanjut dalam sistem kendali?

Jawab:

Machine Learning dapat diintegrasikan lebih lanjut dalam sistem kendali melalui beberapa hal sebagai berikut.

- Model Prediktif

Machine Learning dapat digunakan untuk membangun model prediktif yang memprediksi perilaku sistem berdasarkan data historis. Hal ini dapat memungkinkan pengambilan keputusan yang lebih baik.

- Otomatisasi Proses

Dengan mengintegrasikan Machine Learning, sistem kendali dapat otomatisasi proses pengambilan keputusan, sehingga mampu mengurangi intervensi manusia dan meningkatkan efisiensi.

- Pengendalian Adaptif

Machine Learning memungkinkan sistem untuk beradaptasi secara real-time terhadap perubahan kondisi, seperti fluktuasi dalam input atau gangguan eksternal.

- Optimasi Kerja

Algoritma Machine Learning dapat digunakan untuk mengoptimalkan parameter sistem kendali, sehingga meningkatkan kinerja keseluruhan.

c. Apa saja tantangan dalam penerapan ML dalam sistem real-time?

Jawab:

Tantangan dalam penerapan Machine Learning dalam sistem real-time antara lain:

- Keterbatasan Data

Ketersediaan data yang berkualitas tinggi dan relevan dalam waktu nyata sering kali menjadi tantangan, karena Machine Learning memerlukan data untuk belajar dan beradaptasi.

- Waktu Respons

Sistem real-time memerlukan waktu respon yang cepat dan proses pelatihan model Machine Learning dapat memakan waktu yang dapat menghambat kinerja sistem.

- Kompleksitas Model

Model Machine Learning yang kompleks dapat sulit untuk diinterpretasikan dan diimplementasikan dalam sistem kendali yang memerlukan transparansi dan keandalan.

- Overfitting

Risiko overfitting pada model Machine Learning dapat menyebabkan kinerja yang buruk dalam situasi nyata di mana data baru mungkin berbeda dari data pelatihan.

5. Assignment

Tugas assignment yaitu modifikasi program untuk menggunakan model ML lain (contoh: Decision Tree atau SVM). Kemudian, menambahkan fitur untuk mendeteksi lebih dari satu warna secara simultan. Lalu, eksplorasi dataset lain di Kaggle dan coba implementasikan model pada dataset tersebut. Saya memberikan 2 contoh yaitu dengan model tanpa bounding box dan memakai bounding box. Saya memakai model SVM dengan dataset training_dataset.csv.

Contoh 1 (tanpa bounding box):

```
import os
import cv2
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import joblib
from scipy.special import softmax
```

```
# Path ke dataset
```

```
dataset_folder = 'training_dataset'
```

```
# List untuk menyimpan data
```

```
data = []
```

```
labels = []
```

```
# Looping ke setiap folder warna
```

```
for color_name in os.listdir(dataset_folder):
```

```
    color_path = os.path.join(dataset_folder, color_name)
```

```
    if os.path.isdir(color_path): # Pastikan hanya membaca folder
```

```
        for img_name in os.listdir(color_path):
```

```
            img_path = os.path.join(color_path, img_name)
```

```
            # Baca gambar dalam format BGR dan konversi ke RGB
```

```
            img = cv2.imread(img_path)
```

```
            if img is None:
```

```
                continue # Lewati file yang tidak bisa dibaca
```

```
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # FIXED: Konversi ke RGB
```

```
            # Ambil rata-rata nilai warna (R, G, B)
```

```
            avg_color = img.mean(axis=(0, 1)) # Mean diambil dari semua piksel
```

```
            # Simpan fitur dan label
```

```
            data.append(avg_color)
```

```
            labels.append(color_name)
```

```
# Konversi ke array numpy
```

```
X = np.array(data)
```

```
y = np.array(labels)
```

```
# Normalisasi data
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
# Pilih model SVM
```

```
model = SVC(kernel='linear', random_state=42, probability=True) # Mengatur
```

```
probability=True
```

```
model.fit(X_train, y_train)
```

```
# Evaluasi model
```

```
train_acc = accuracy_score(y_train, model.predict(X_train))
```

```
test_acc = accuracy_score(y_test, model.predict(X_test))
```

```
print(f"Akurasi pada data latih: {train_acc * 100:.2f}%")
```

```
print(f"Akurasi pada data uji: {test_acc * 100:.2f}%")
```

```
# Simpan model dan scaler
```

```
joblib.dump(model, 'color_model.pkl')
```

```
joblib.dump(scaler, 'scaler.pkl')
```

```
print("Model dan scaler berhasil disimpan!")
```

```
# Muat kembali model dan scaler untuk deteksi warna
```

```
model = joblib.load('color_model.pkl')
```

```
scaler = joblib.load('scaler.pkl')
```

```
# Inisialisasi kamera
```

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
if not ret:
```

```
    break
```

```
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # FIXED: Konversi ke RGB
```

```
# Ambil pixel tengah gambar
```

```
height, width, _ = frame.shape
```

```
pixel_center = frame[height // 2, width // 2]
```

```
# Pastikan bentuk data sesuai untuk StandardScaler
```

```
pixel_center_scaled = scaler.transform(pixel_center.reshape(1, -1))
```

```
# Prediksi warna dan skor kepercayaan
```

```
color_pred = model.predict(pixel_center_scaled)[0]
```

```
conf_scores = model.predict_proba(pixel_center_scaled)[0]
```

```
conf = conf_scores[np.argmax(conf_scores)]
```

```
# Tampilkan warna dan skor kepercayaan pada frame
```

```
label = f"{color_pred}: {conf:.2f}"
```

```
cv2.putText(frame, label, (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)
```

```
# Konversi kembali ke BGR untuk tampilan OpenCV
```

```
cv2.imshow('Frame', cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Penjelasan: Import Libraries

```
import os  
  
import cv2  
  
import numpy as np  
  
import pandas as pd  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.preprocessing import StandardScaler  
  
from sklearn.svm import SVC  
  
from sklearn.metrics import accuracy_score  
  
import joblib
```

os: Digunakan untuk berinteraksi dengan sistem file, seperti membaca folder dan file.

cv2: Library OpenCV untuk pemrosesan gambar dan video.

numpy: Digunakan untuk operasi numerik dan manipulasi array.

pandas: Digunakan untuk manipulasi dan analisis data, terutama untuk membaca dan mengelola dataset.

sklearn.model_selection.train_test_split: Digunakan untuk membagi dataset menjadi data pelatihan dan pengujian.

sklearn.preprocessing.StandardScaler: Digunakan untuk menstandarisasi fitur dengan menghilangkan rata-rata dan menskalakan ke unit varians.

sklearn.svm.SVC: Model Support Vector Classifier untuk klasifikasi.

sklearn.metrics.accuracy_score: Digunakan untuk menghitung akurasi model.

joblib: Digunakan untuk menyimpan dan memuat model yang telah dilatih.

Path ke Dataset

```
dataset_folder = 'training_dataset'
```

dataset_folder: Variabel yang menyimpan path ke folder yang berisi dataset. Anda perlu memastikan folder ini ada dan berisi data yang diperlukan.

List untuk Menyimpan Data

```
data = []
```

```
labels = []
```

data: List untuk menyimpan fitur (misalnya, nilai warna) dari gambar.

labels: List untuk menyimpan label (nama warna) yang sesuai dengan fitur.

Looping ke Setiap Folder Warna

```
for color_name in os.listdir(dataset_folder):
```

```
    color_path = os.path.join(dataset_folder, color_name)
```

```
    if os.path.isdir(color_path): # Pastikan hanya membaca folder
```

```
        for img_name in os.listdir(color_path):
```

```
            img_path = os.path.join(color_path, img_name)
```

os.listdir(dataset_folder): Mengambil daftar semua folder (nama warna) dalam folder dataset.

os.path.join: Menggabungkan path untuk membentuk path lengkap ke folder warna.

os.path.isdir: Memeriksa apakah path yang diberikan adalah folder.

Baca Gambar dan Ekstrak Fitur

```
img = cv2.imread(img_path)
```

```
if img is None:
```

```
    continue # Lewati file yang tidak bisa dibaca
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Konversi ke RGB
```

```
# Ambil rata-rata nilai warna (R, G, B)
```

```
avg_color = img.mean(axis=(0, 1)) # Mean diambil dari semua piksel
```

```
# Simpan fitur dan label
```

```
data.append(avg_color)
```

```
labels.append(color_name)
```

`cv2.imread`: Membaca gambar dari path yang diberikan.

`img is None`: Memeriksa apakah gambar berhasil dibaca. Jika tidak, lanjutkan ke iterasi berikutnya.

`cv2.cvtColor`: Mengonversi gambar dari format BGR (default OpenCV) ke RGB.

`img.mean(axis=(0, 1))`: Menghitung rata-rata nilai warna (R, G, B) dari gambar.

`data.append(avg_color)`: Menyimpan rata-rata warna ke dalam list data.

`labels.append(color_name)`: Menyimpan nama warna ke dalam list labels.

Konversi ke Array Numpy

```
X = np.array(data)
```

```
y = np.array(labels)
```

`np.array(data)`: Mengonversi list data menjadi array NumPy untuk digunakan dalam pelatihan model.

`np.array(labels)`: Mengonversi list labels menjadi array NumPy.

Normalisasi Data

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

`StandardScaler()`: Membuat objek scaler untuk menstandarisasi data.

`scaler.fit_transform(X)`: Menghitung rata-rata dan deviasi standar dari data, lalu menstandarisasi data.

Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

`train_test_split`: Membagi dataset menjadi data pelatihan (80%) dan data pengujian (20%).

Model SVM

```
model = SVC(kernel='linear', random_state=42, probability=True)
```

```
model.fit(X_train, y_train)
```

SVC: Membuat model SVM dengan kernel linear.

model.fit: Melatih model menggunakan data pelatihan.

Evaluasi Model

```
train_acc = accuracy_score(y_train, model.predict(X_train))
```

```
test_acc = accuracy_score(y_test, model.predict(X_test))
```

```
print(f"Akurasi pada data latih: {train_acc * 100:.2f}%")
```

```
print(f"Akurasi pada data uji: {test_acc * 100:.2f}%")
```

accuracy_score: Menghitung akurasi model pada data pelatihan dan pengujian.

print: Menampilkan akurasi dalam format persen.

Simpan Model dan Scaler

```
joblib.dump(model, 'color_model.pkl')
```

```
joblib.dump(scaler, 'scaler.pkl')
```

```
print("Model dan scaler berhasil disimpan!")
```

joblib.dump: Menyimpan model dan scaler ke file untuk digunakan di masa depan.

Muat Kembali Model dan Scaler untuk Deteksi Warna

```
model = joblib.load('color_model.pkl')
```

```
scaler = joblib.load('scaler.pkl')
```

joblib.load: Memuat model dan scaler yang telah disimpan sebelumnya.

Inisialisasi Kamera

```
cap = cv2.VideoCapture(0)
```

cv2.VideoCapture: Menginisialisasi kamera untuk menangkap video.

Loop untuk Deteksi Warna

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # Konversi ke RGB
```

```
# Ambil pixel tengah gambar
```

```
height, width, _ = frame.shape
```

```
pixel_center = frame[height // 2, width // 2]
```

```
# Pastikan bentuk data sesuai untuk StandardScaler
```

```
pixel_center_scaled = scaler.transform(pixel_center.reshape(1, -1))
```

```
# Prediksi warna dan skor kepercayaan
```

```
color_pred = model.predict(pixel_center_scaled)[0]
```

```
conf_scores = model.predict_proba(pixel_center_scaled)[0]
```

```
conf = conf_scores[np.argmax(conf_scores)]
```

```
# Tampilkan warna dan skor kepercayaan pada frame
```

```
label = f"{color_pred}: {conf:.2f}"
```

```
cv2.putText(frame, label, (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)
```

```
# Konversi kembali ke BGR untuk tampilan OpenCV
```

```
cv2.imshow('Frame', cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

while True: Loop untuk terus menangkap video.

cap.read(): Membaca frame dari kamera.

cv2.cvtColor: Mengonversi frame ke format RGB.

pixel_center: Mengambil pixel tengah dari frame.

scaler.transform: Mengubah bentuk pixel agar sesuai untuk scaler.

model.predict: Memprediksi warna berdasarkan pixel yang telah di-scale.

model.predict_proba: Menghitung probabilitas untuk setiap kelas.

cv2.putText: Menampilkan hasil prediksi dan confidence score pada frame.

cv2.imshow: Menampilkan frame dengan hasil deteksi.

cv2.waitKey: Menunggu input dari keyboard untuk keluar dari loop.

Contoh 2 (Memakai Bounding Box):

```
import os
```

```
import cv2
```

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score
```

```
import joblib
```

```
# Path ke dataset
```

```
dataset_folder = 'training_dataset'
```

```
# List untuk menyimpan data
```

```
data = []
```

```
labels = []
```

```
# Looping ke setiap folder warna
```

```
for color_name in os.listdir(dataset_folder):
```

```
    color_path = os.path.join(dataset_folder, color_name)
```

```
    if os.path.isdir(color_path): # Pastikan hanya membaca folder
```

```
        for img_name in os.listdir(color_path):
```

```
            img_path = os.path.join(color_path, img_name)
```

```
            # Baca gambar dalam format BGR dan konversi ke RGB
```

```
            img = cv2.imread(img_path)
```

```
            if img is None:
```

```
                continue # Lewati file yang tidak bisa dibaca
```

```
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # FIXED: Konversi ke RGB
```

```
            # Ambil rata-rata nilai warna (R, G, B)
```

```
            avg_color = img.mean(axis=(0, 1)) # Mean diambil dari semua piksel
```

```
            # Simpan fitur dan label
```

```
            data.append(avg_color)
```

```
            labels.append(color_name)
```

```
# Konversi ke array numpy
```

```
X = np.array(data)
```

```
y = np.array(labels)
```

```
# Normalisasi data
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
# Pilih model SVM
```

```
model = SVC(kernel='linear', random_state=42, probability=True) # Mengatur
```

```
probability=True
```

```
model.fit(X_train, y_train)
```

```
# Evaluasi model
```

```
train_acc = accuracy_score(y_train, model.predict(X_train))
```

```
test_acc = accuracy_score(y_test, model.predict(X_test))
```

```
print(f"Akurasi pada data latih: {train_acc * 100:.2f}%")
```

```
print(f"Akurasi pada data uji: {test_acc * 100:.2f}%")
```

```
# Simpan model dan scaler
```

```
joblib.dump(model, 'color_model.pkl')
```

```
joblib.dump(scaler, 'scaler.pkl')
```

```
print("Model dan scaler berhasil disimpan!")
```

```
# Muat kembali model dan scaler untuk deteksi warna
```

```
model = joblib.load('color_model.pkl')
```

```
scaler = joblib.load('scaler.pkl')
```

```
# Inisialisasi kamera
```

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

if not ret:

break

frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # Konversi ke RGB

Normalisasi seluruh frame

frame_scaled = scaler.transform(frame_rgb.reshape(-1, 3))

Prediksi warna untuk seluruh frame

color_probs = model.predict_proba(frame_scaled)

Hitung persentase setiap warna

color_counts = np.sum(color_probs, axis=0)

total_pixels = frame.shape[0] * frame.shape[1]

color_percentages = (color_counts / total_pixels) * 100

List untuk menyimpan bounding boxes dengan confidence score

bounding_boxes = []

Tampilkan hasil dan gambar bounding box

for i, color in enumerate(model.classes_):

conf = color_percentages[i]

if conf > 0: # Hanya tampilkan warna yang terdeteksi

Buat mask untuk warna yang terdeteksi

mask = (color_probs[:, i] > 0.5).reshape(frame.shape[0], frame.shape[1]) # Threshold

if np.any(mask):

Temukan kontur dari mask

contours, _ = cv2.findContours(mask.astype(np.uint8), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

for contour in contours:

if cv2.contourArea(contour) > 100: # Filter kontur kecil

x, y, w, h = cv2.boundingRect(contour)

```

        bounding_boxes.append((x, y, w, h, conf, color)) # Simpan bounding box
dengan confidence score

# Sortir bounding boxes berdasarkan confidence score (descending)
bounding_boxes.sort(key=lambda x: x[4], reverse=True)

# Pilih hanya top 2 bounding boxes
top_2_bounding_boxes = bounding_boxes[:2]

# Gambar hanya top 2 bounding boxes
for x, y, w, h, conf, color in top_2_bounding_boxes:
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2) # Gambar bounding box
    label = f"{color}: {conf:.2f}%"
    cv2.putText(frame, label, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

# Tampilkan frame dengan bounding box
cv2.imshow('Frame', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Penjelasan:

Import Libraries

```

import os

import cv2

import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score
```

```
import joblib
```

os: Digunakan untuk berinteraksi dengan sistem file, seperti membaca folder dan file.

cv2: Library OpenCV untuk pemrosesan gambar dan video.

numpy: Digunakan untuk operasi numerik dan manipulasi array.

pandas: Digunakan untuk manipulasi dan analisis data, terutama untuk bekerja dengan DataFrame.

sklearn.model_selection.train_test_split: Digunakan untuk membagi dataset menjadi data pelatihan dan pengujian.

sklearn.preprocessing.StandardScaler: Digunakan untuk menstandarisasi fitur dengan menghapus rata-rata dan menskalakan ke unit varians.

sklearn.svm.SVC: Kelas untuk Support Vector Classification.

sklearn.metrics.accuracy_score: Digunakan untuk menghitung akurasi model.

joblib: Digunakan untuk menyimpan dan memuat model yang telah dilatih.

Path ke Dataset

```
dataset_folder = 'training_dataset'
```

dataset_folder: Variabel ini menyimpan path ke folder yang berisi dataset. Anda harus memastikan bahwa folder ini ada dan berisi data yang diperlukan.

List untuk Menyimpan Data

```
data = []
```

```
labels = []
```

data: List ini akan menyimpan fitur (misalnya, nilai warna) dari gambar.

labels: List ini akan menyimpan label (nama warna) yang sesuai dengan setiap gambar.

Looping ke Setiap Folder Warna

```
for color_name in os.listdir(dataset_folder):
```

```
    color_path = os.path.join(dataset_folder, color_name)
```

```
    if os.path.isdir(color_path): # Pastikan hanya membaca folder
```

```
        for img_name in os.listdir(color_path):
```

```
            img_path = os.path.join(color_path, img_name)
```

```
            # Baca gambar dalam format BGR dan konversi ke RGB
```

```
            img = cv2.imread(img_path)
```

```
            if img is None:
```

```
                continue # Lewati file yang tidak bisa dibaca
```

```
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Konversi ke RGB
```

```
            # Ambil rata-rata nilai warna (R, G, B)
```

```
            avg_color = img.mean(axis=(0, 1)) # Mean diambil dari semua piksel
```

```
# Simpan fitur dan label
```

```
data.append(avg_color)
```

```
labels.append(color_name)
```

`os.listdir(dataset_folder)`: Mengambil daftar semua folder (nama warna) dalam folder dataset.

`os.path.join`: Menggabungkan path untuk membentuk path lengkap ke folder warna.

`os.path.isdir`: Memeriksa apakah path yang diberikan adalah folder.

`cv2.imread`: Membaca gambar dari path yang diberikan.

`cv2.cvtColor`: Mengonversi gambar dari format BGR (yang digunakan OpenCV) ke RGB.

`img.mean(axis=(0, 1))`: Menghitung rata-rata nilai warna (R, G, B) dari gambar dengan mengambil rata-rata dari semua piksel.

`data.append(avg_color)`: Menambahkan rata-rata warna ke list data.

`labels.append(color_name)`: Menambahkan nama warna ke list labels.

Konversi ke Array Numpy

```
X = np.array(data)
```

```
y = np.array(labels)
```

X: Array numpy yang berisi fitur (rata-rata warna).

y: Array numpy yang berisi label (nama warna).

Normalisasi Data

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

StandardScaler: Membuat objek scaler untuk menstandarisasi data.

fit_transform: Menghitung rata-rata dan deviasi standar dari data, lalu menerapkannya untuk menstandarisasi data.

Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

train_test_split: Membagi dataset menjadi data pelatihan dan pengujian. Di sini, 20% dari data digunakan untuk pengujian.

Model SVM

```
model = SVC(kernel='linear', random_state=42, probability=True)
```

```
model.fit(X_train, y_train)
```

SVC: Membuat model SVM dengan kernel linear.

fit: Melatih model menggunakan data pelatihan.

Evaluasi Model

```
train_acc = accuracy_score(y_train, model.predict(X_train))
```

```
test_acc = accuracy_score(y_test, model.predict(X_test))
```

```
print(f"Akurasi pada data latih: {train_acc * 100:.2f}%")
```

```
print(f"Akurasi pada data uji: {test_acc * 100:.2f}%")
```

accuracy_score: Menghitung akurasi model pada data pelatihan dan pengujian.

print: Menampilkan akurasi dalam format persen.

Simpan Model dan Scaler

```
joblib.dump(model, 'color_model.pkl')
```

```
joblib.dump(scaler, 'scaler.pkl')
```

```
print("Model dan scaler berhasil disimpan!")
```

joblib.dump: Menyimpan model dan scaler ke file untuk digunakan di masa depan.

Muat Kembali Model dan Scaler untuk Deteksi Warna

```
model = joblib.load('color_model.pkl')
```

```
scaler = joblib.load('scaler.pkl')
```

joblib.load: Memuat model dan scaler yang telah disimpan sebelumnya.

Inisialisasi Kamera

```
cap = cv2.VideoCapture(0)
```

cv2.VideoCapture: Menginisialisasi kamera untuk menangkap video.

Loop untuk Deteksi Warna

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # Konversi ke RGB

# Ambil pixel tengah gambar

height, width, _ = frame.shape

pixel_center = frame[height // 2, width // 2]

# Pastikan bentuk data sesuai untuk StandardScaler

pixel_center_scaled = scaler.transform(pixel_center.reshape(1, -1))

# Prediksi warna dan skor kepercayaan

color_pred = model.predict(pixel_center_scaled)[0]

conf_scores = model.predict_proba(pixel_center_scaled)[0]

conf = conf_scores[np.argmax(conf_scores)]

# Tampilkan warna dan skor kepercayaan pada frame

label = f"{color_pred}: {conf:.2f}"

cv2.putText(frame, label, (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)

# Konversi kembali ke BGR untuk tampilan OpenCV

cv2.imshow('Frame', cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))

if cv2.waitKey(1) & 0xFF == ord('q'):

    break
```

cap.release()

cv2.destroyAllWindows()

while True: Loop untuk terus menangkap video.

cap.read(): Membaca frame dari kamera.

cv2.cvtColor: Mengonversi frame ke format RGB.

pixel_center: Mengambil pixel tengah dari frame.

scaler.transform: Mengubah bentuk pixel agar sesuai untuk scaler.

model.predict: Memprediksi warna berdasarkan pixel yang telah di-scale.

model.predict_proba: Menghitung probabilitas untuk setiap kelas.

cv2.putText: Menampilkan hasil prediksi dan confidence score pada frame.

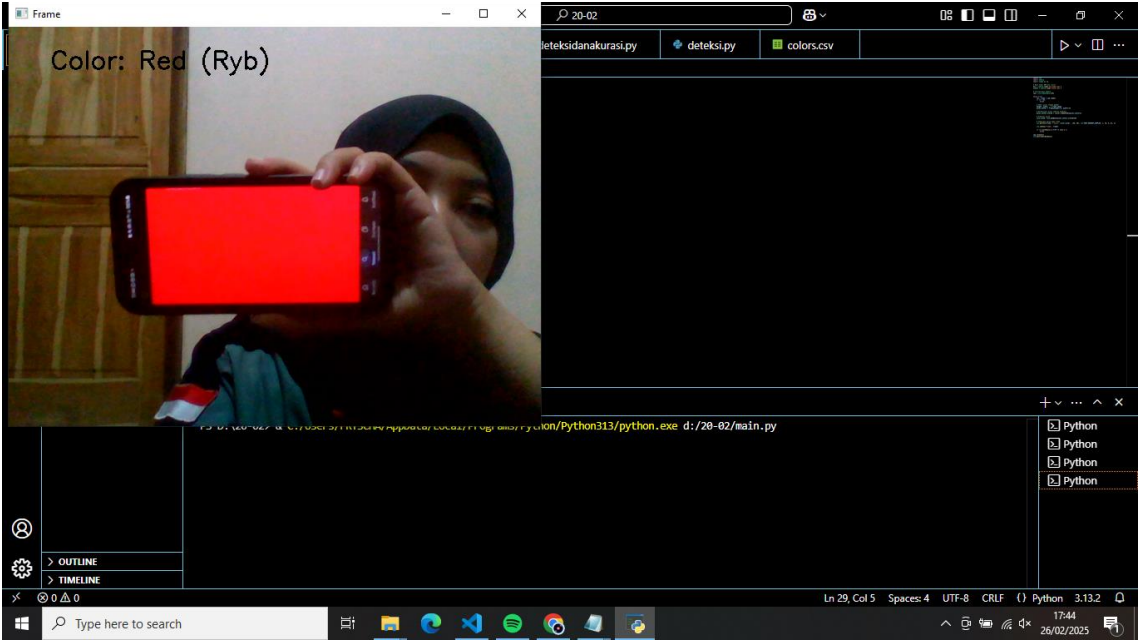
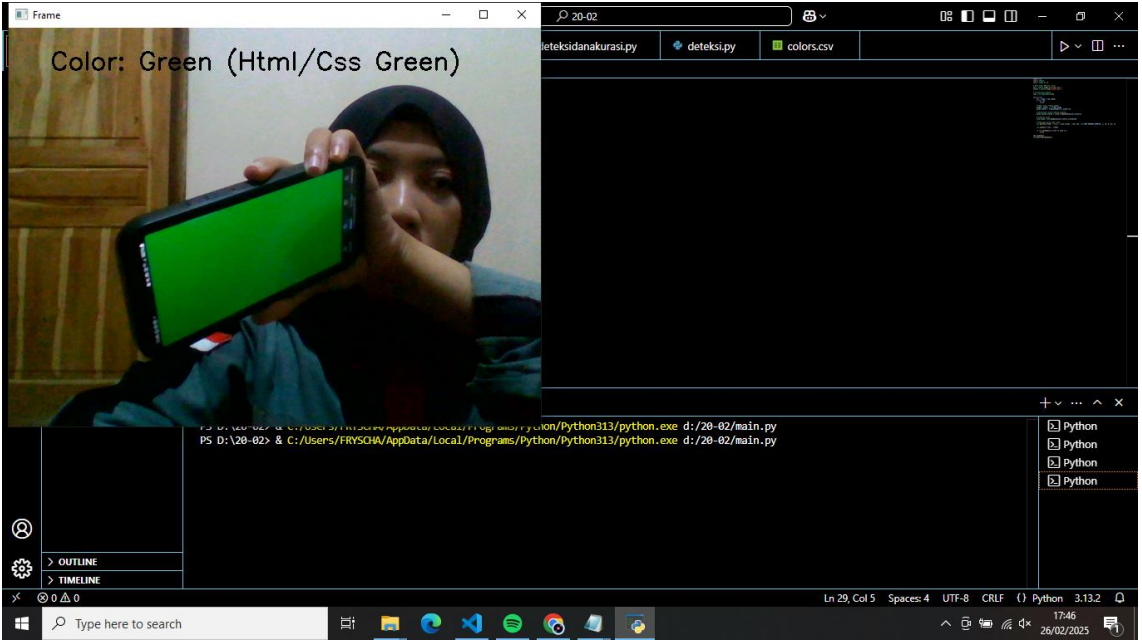
cv2.imshow: Menampilkan frame dengan hasil deteksi.

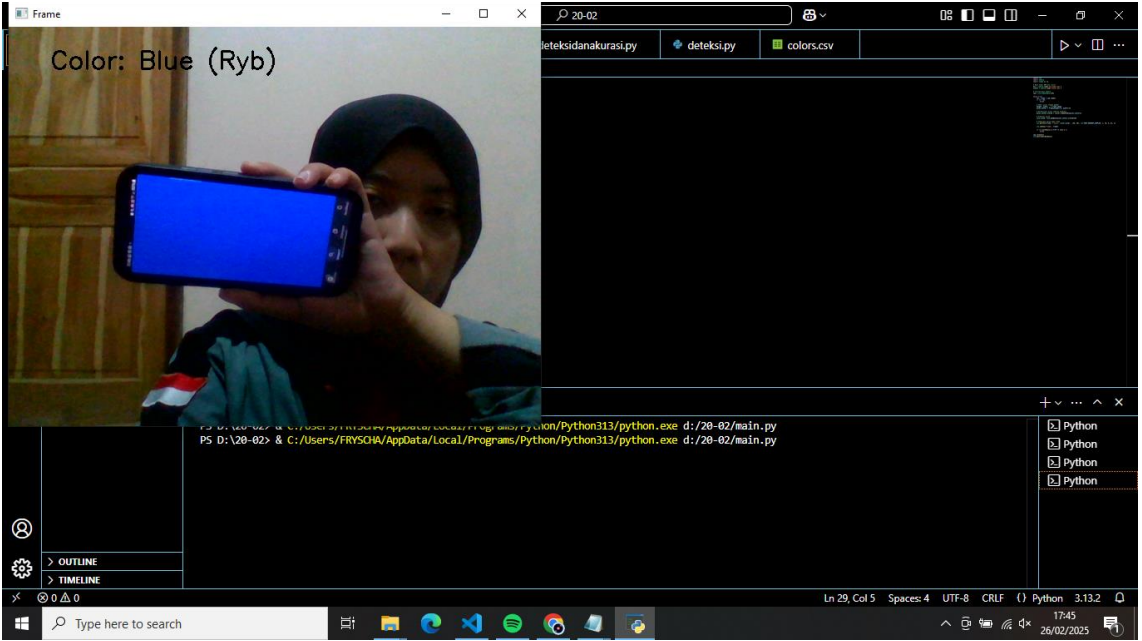
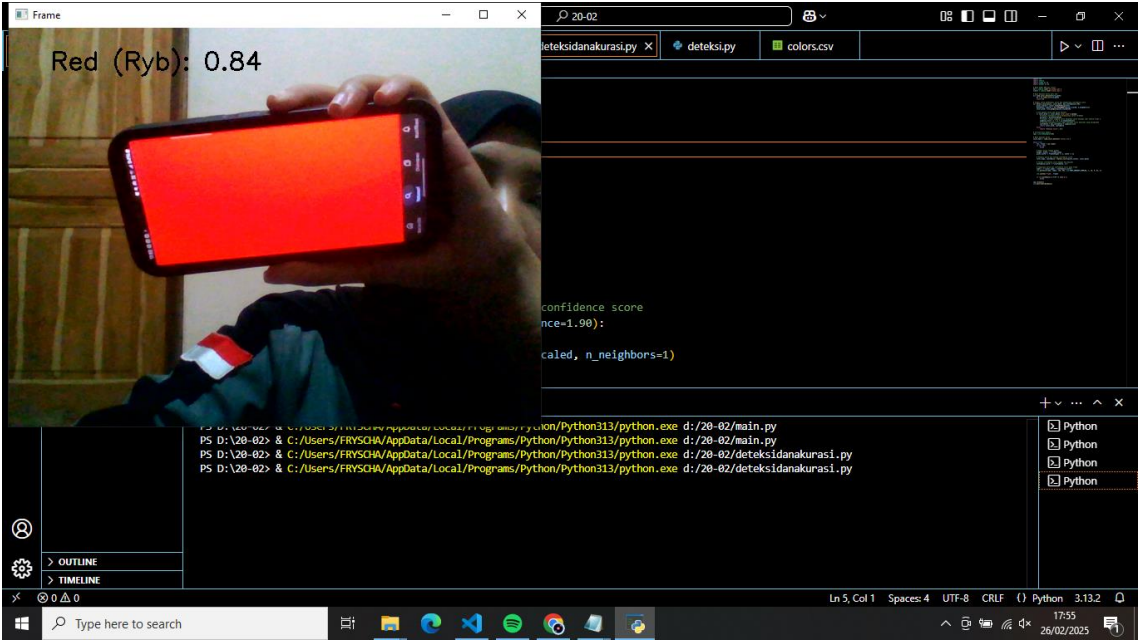
cap.release() dan cv2.destroyAllWindows(): Menutup kamera dan jendela OpenCV saat selesai.

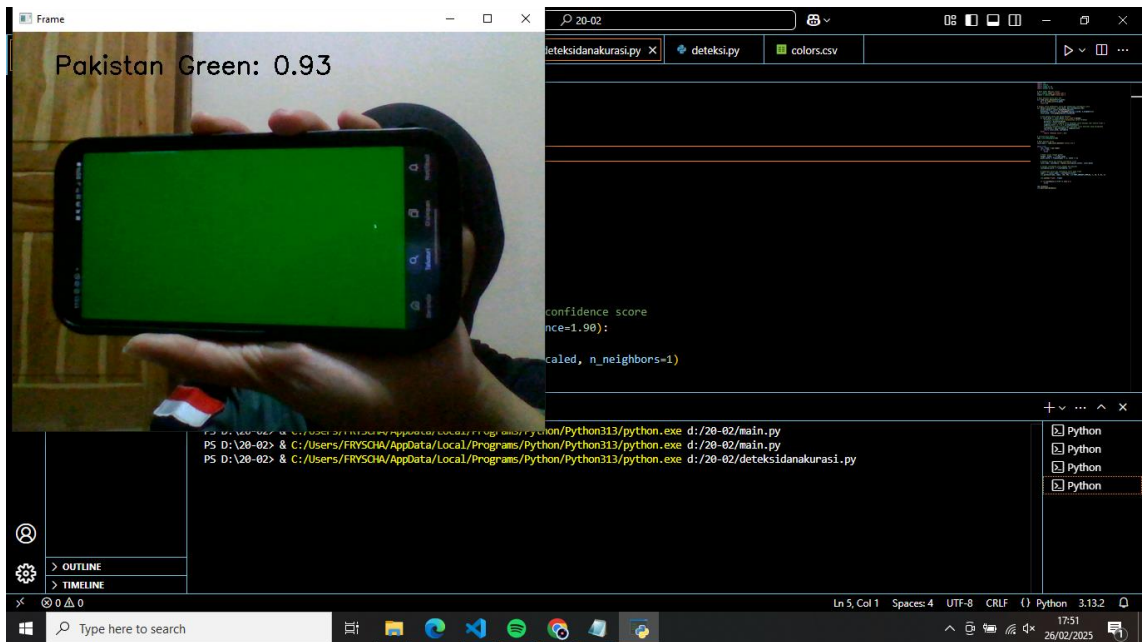
6. Data dan Output Hasil Pengamatan

Sajikan data dan hasil yang diperoleh selama percobaan. Gunakan tabel untuk menyajikan data jika diperlukan.

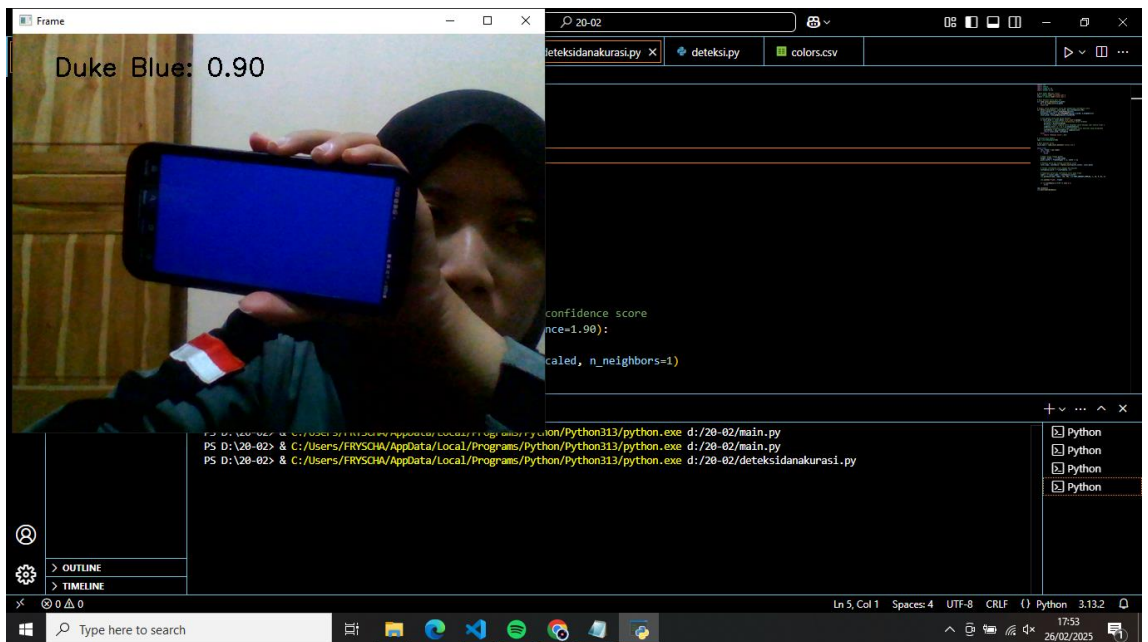
No	Variabel	Hasil Pengamatan
1	Warna Merah	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna merah terdeteksi.

		
2	Warna Hijau	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna hijau terdeteksi. 
3	Warna Biru	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna biru terdeteksi.

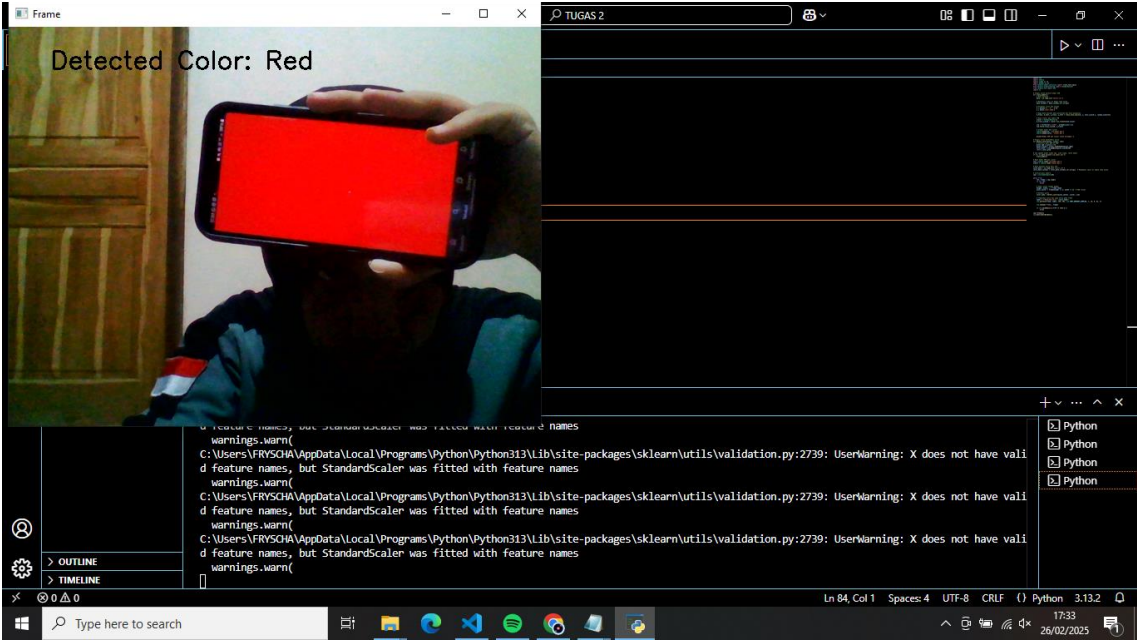
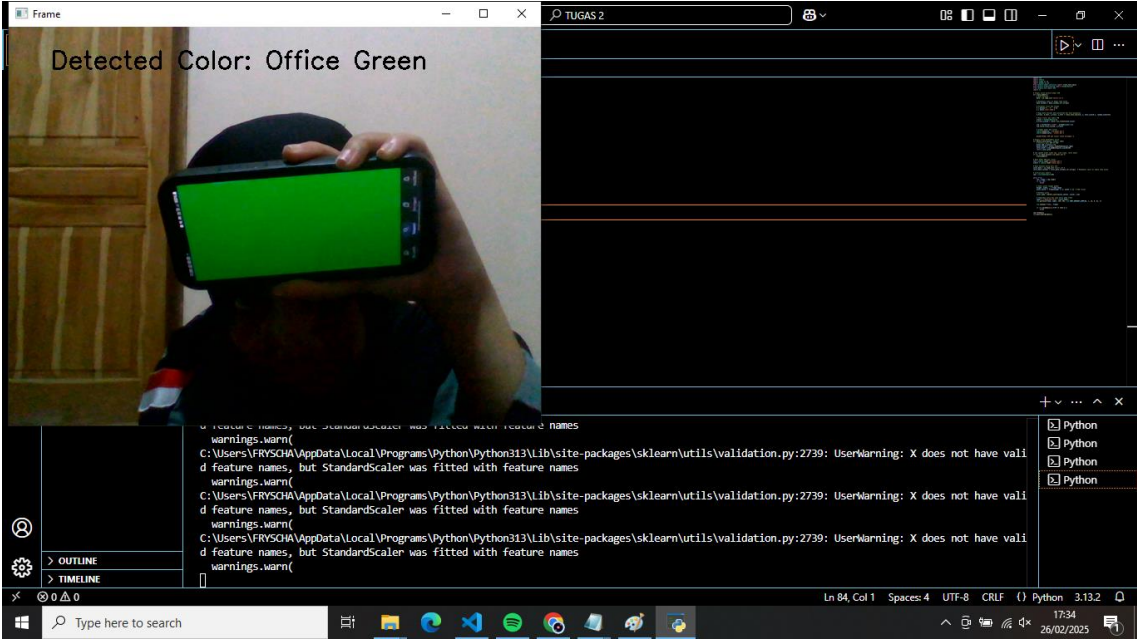
		
4	Warna Merah	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna merah terdeteksi dengan akurasi 0.84. 
5	Warna Hijau	Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna hijau terdeteksi dengan akurasi 0.93.

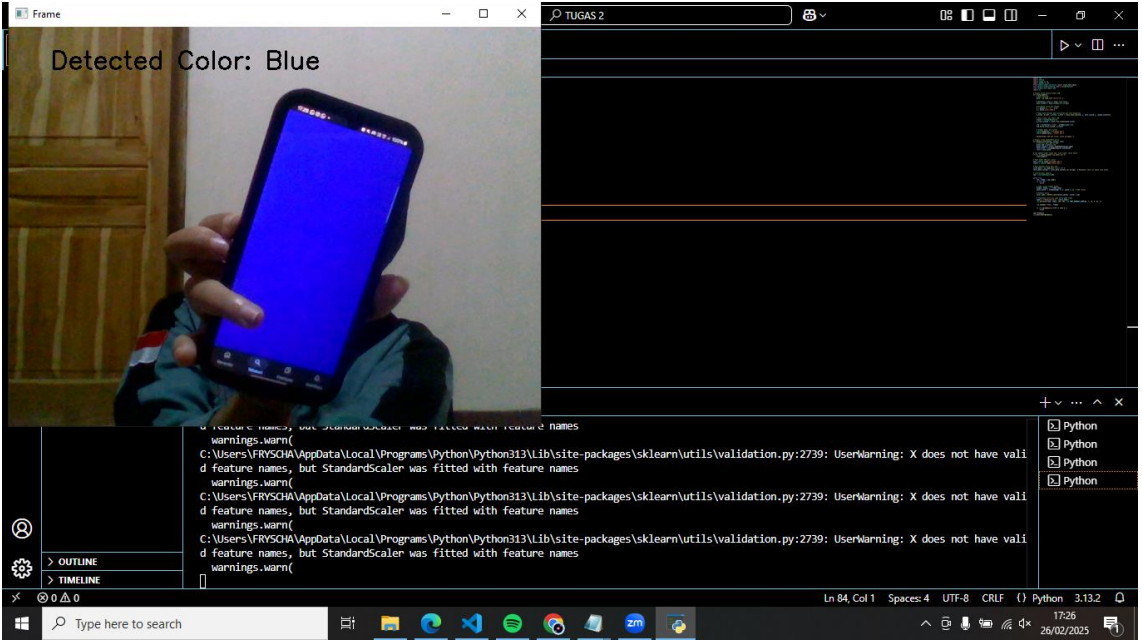
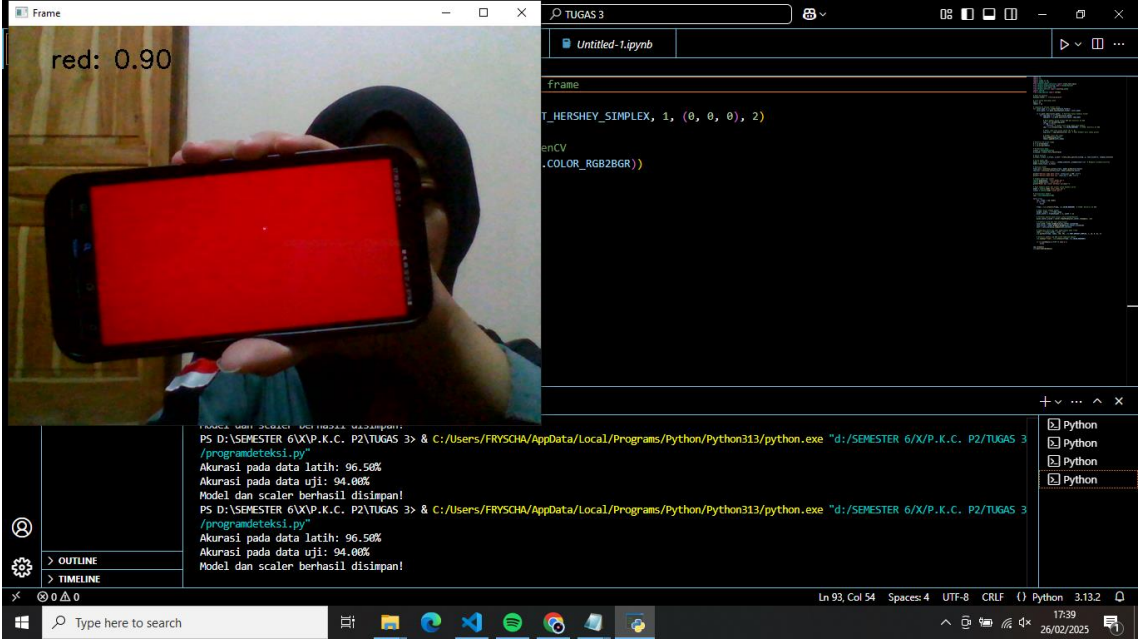


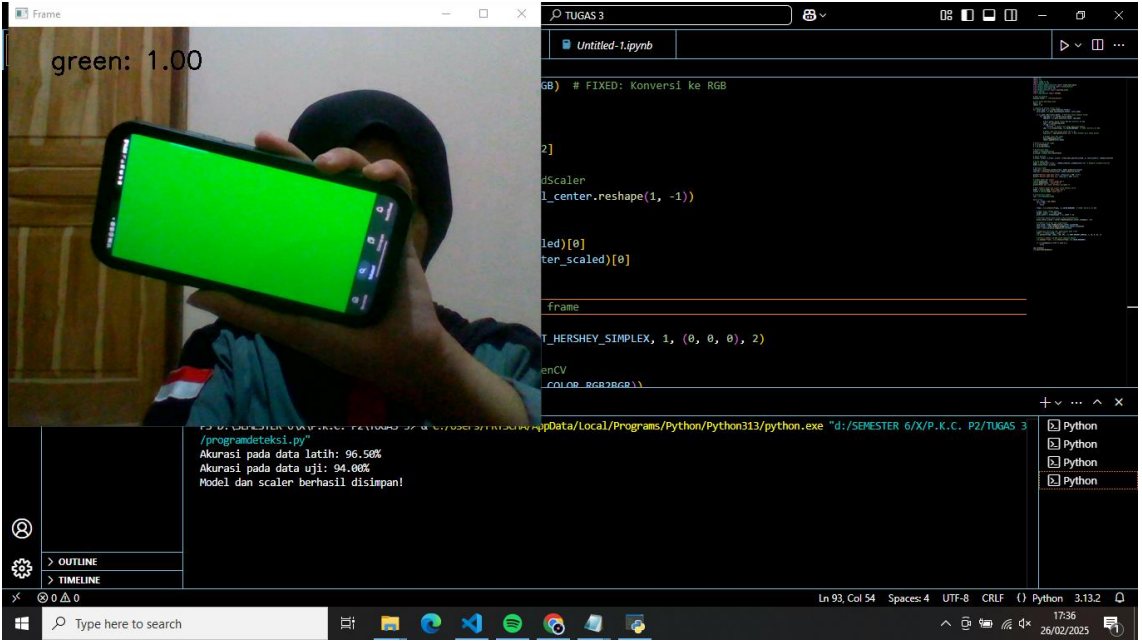
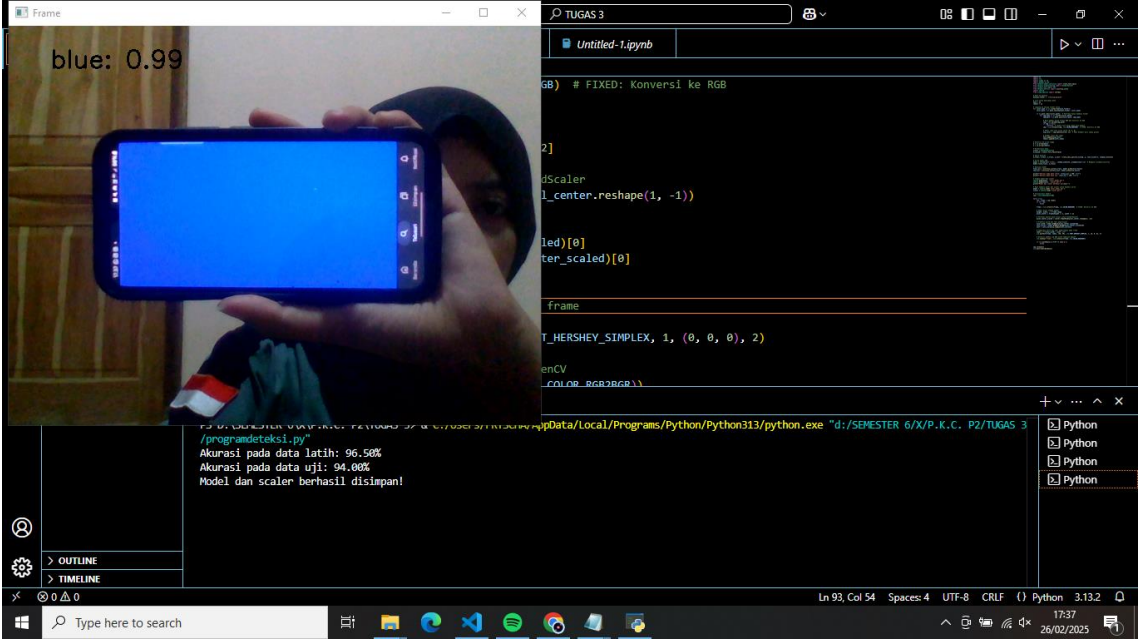
6 Warna Biru Output dari program Machine Learning (ML) KNN menunjukkan bahwa adanya warna biru terdeteksi dengan akurasi 0.90.



7 Warna Merah Output dari program Machine Learning (ML) SVM dengan dataset colors.csv menunjukkan bahwa adanya warna merah terdeteksi.

		
8	Warna Hijau	Output dari program Machine Learning (ML) SVM dengan dataset colors.csv menunjukkan bahwa adanya warna hijau terdeteksi. 
9	Warna Biru	Output dari program Machine Learning (ML) SVM dengan dataset colors.csv menunjukkan bahwa adanya warna biru terdeteksi.

		
10	Warna Merah	Output dari program Machine Learning (ML) SVM dengan dataset training_dataset.csv menunjukkan bahwa adanya warna merah terdeteksi dengan akurasi 0.90. 
11	Warna Hijau	Output dari program Machine Learning (ML) SVM dengan dataset training_dataset.csv menunjukkan bahwa adanya warna hijau terdeteksi dengan akurasi 1.00.

		 The screenshot displays a Windows environment. On the left, a video frame shows a person holding a smartphone with a bright green screen. Text overlay on the frame reads "green: 1.00". On the right, a Python IDE (Untitled-1.py) shows code for color detection. The code includes comments like "# FIXED: Konversi ke RGB" and "# FIXED: Konversi ke HSV". The output window at the bottom shows the results: "Akurasi pada data latih: 96.50%", "Akurasi pada data uji: 94.00%", and "Model dan scaler berhasil disimpan!".
12	Warna Biru	Output dari program Machine Learning (ML) SVM dengan dataset training_dataset.csv menunjukkan bahwa adanya warna biru terdeteksi dengan akurasi 0.99.  The screenshot displays a Windows environment. On the left, a video frame shows a person holding a smartphone with a bright blue screen. Text overlay on the frame reads "blue: 0.99". On the right, a Python IDE (Untitled-1.py) shows code for color detection. The code includes comments like "# FIXED: Konversi ke RGB" and "# FIXED: Konversi ke HSV". The output window at the bottom shows the results: "Akurasi pada data latih: 96.50%", "Akurasi pada data uji: 94.00%", and "Model dan scaler berhasil disimpan!".
13	Multiple	Output dari program Machine Learning (ML) SVM dengan dataset training_dataset.csv menunjukkan bahwa adanya berbagai warna terdeteksi dengan akurasi berformat persen.

Santoso J.T., 2023. *KECERDASAN BUATAN (Artificial Intelligence)*. Yayasan Prima Agus Teknik
Bekerja sama dengan Universitas Sains & Teknologi Komputer (Universitas STEKOM).
Semarang.

Siregar, M. H., & Mulyana, D. I. (2024). Teknologi Artificial Intelligence (AI) Vision Swift dalam
Sistem Pemantauan Latihan Bulu Tangkis dengan Algoritma Optical Flow. *Jurnal
Indonesia: Manajemen Informatika dan Komunikasi*, 5(3), 3349–3361.
<https://doi.org/10.35870/jimik.v5i3.1027>